

--- Arquivo: 5.1-SQL.md ---

```
### 5.1. `SQL`

CREATE DATABASE estude43_estacao_db

CREATE TABLE leituras_sensores (
  id INT AUTO_INCREMENT PRIMARY KEY,
  estacao_id VARCHAR(20) NOT NULL DEFAULT 'ESTACAO_01',
  temperatura FLOAT(4,1) NOT NULL,
  umidade FLOAT(4,1) NOT NULL,
  contador INT NOT NULL,
  energia_ok TINYINT(1) NOT NULL,
  aquecedor_ligado TINYINT(1) NOT NULL DEFAULT 0,
  exaustao_ligada TINYINT(1) NOT NULL,
  exaustao_ok TINYINT(1) NOT NULL,
  data_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE comandos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  acionar_aquecedor TINYINT(1) NOT NULL DEFAULT 0,
  ligar_exaustao TINYINT(1) NOT NULL DEFAULT 0,
  executando TINYINT(1) NOT NULL DEFAULT 0,
  data_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

--- Arquivo: 5.2-Platformio_ini.md ---

```
### 5.2. `platformio.ini`

[env]
platform = espressif32
board = esp32dev
framework = arduino
monitor_speed = 115200
board_build.filesystem = littlefs

# CONFIGURAÇÃO DA PLACA EMISSORA

[env:ttgo_emissor]
upload_port = COM4
monitor_port = COM4
build_src_filter = +<\\*> -<receptor.cpp> -<main.cpp>
```

```

lib_deps =
adafruit/DHT sensor library
adafruit/Adafruit Unified Sensor
adafruit/Adafruit GFX Library
adafruit/Adafruit SSD1306

# CONFIGURAÇÃO DA PLACA RECEPTORA

[env:ttgo_receptor]
upload_port = COM3
monitor_port = COM3
build_src_filter = +<*\*> -<emissor.cpp> -<main.cpp>
lib_deps =
adafruit/Adafruit GFX Library
adafruit/Adafruit SSD1306
https://github.com/me-no-dev/ESPAsyncWebServer.git
https://github.com/me-no-dev/AsyncTCP.git

```

--- Arquivo: 5.3-emissor_cpp.md ---

```

### 5.3. `src/emissor.cpp`

#include <esp_now.h>
#include <esp_wifi.h>
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT.h>
#include <SPI.h>
#include <LoRa.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

#define DHTPIN 4
#define DHTTYPE DHT11
DHT dht(DHTPIN, DHTTYPE);

#define PINO_AQUECEDOR 23
#define PINO_EXAUSTAO 25
#define PINO_RETORNO_ENERGIA 34
#define PINO_RETORNO_EXAUSTAO 35

unsigned long tempoInicioAquecedor = 0;
bool aquecedorAtivo = false;
const unsigned long DURACAO_AQUECEDOR = 30000; // 30s

```

```

uint8_t macReceptor[] = {0xA0, 0xDD, 0x6C, 0x67, 0xC6, 0x34}; // MAC do Receptor

typedef struct struct_mensagem {
    int contador;
    float temperatura;
    float umidade;
    bool energiaOk;
    bool exaustaoOK;
} struct_mensagem;

typedef struct struct_comando {
    bool acionarAquecedor;
    bool ligarExaustao;
} struct_comando;

struct_mensagem dadosEnvio;
struct_comando comandoRecebido;
esp_now_peer_info_t peerInfo;

int contadorPacotes = 0;
unsigned long ultimoEnvio = 0;

void AoReceberComando(const uint8_t *mac, const uint8_t *incomingData, int len) {
    memcpy(&comandoRecebido, incomingData, sizeof(comandoRecebido));

    //---imprime serial confirma recebimento comando do db comandos
    Serial.println("=== COMANDO RECEBIDO ===");
    Serial.print("Aquecedor: ");
    Serial.println(comandoRecebido.acionarAquecedor);

    Serial.print("Exaustao: ");
    Serial.println(comandoRecebido.ligarExaustao);
    //-----

    if (comandoRecebido.acionarAquecedor){
        digitalWrite(PINO_AQUECEDOR, HIGH);

        //diagnostico botÃ£o, deletar
        Serial.print("GPIO aquecedor apos HIGH = ");
        Serial.println(digitalRead(PINO_AQUECEDOR));

        aquecedorAtivo = true;
        tempoInicioAquecedor = millis();
        Serial.println(">>> Aquecedor Ligado via ESP-NOW! <<<");
        //diagnostico ligamento:
        Serial.print("aquecedorAtivo = ");
        Serial.println(aquecedorAtivo);
        Serial.print("tempoInicioAquecedor = ");
        Serial.println(tempoInicioAquecedor);
        //-----
    }
    Serial.print("Comando Exaustao recebido: ");
    Serial.println(comandoRecebido.ligarExaustao);

```

```
    digitalWrite(PINO_EXAUSTAO, comandoRecebido.ligarExaustao ? HIGH : LOW);
}

#define LORA_SS    18
#define LORA_RST    14
#define LORA_DIO0    26

void setup() {
    Serial.begin(115200);

    SPI.begin(5, 19, 27, 18);
    LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);
    if (!LoRa.begin(915E6))
    {
        Serial.println("Falha ao iniciar LoRa");
        while (true);
    }
    Serial.println("LoRa iniciado com sucesso");

    // Inicializa  o do OLED no Emissor
    Wire.begin(21, 22);
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    display.setTextColor(WHITE);
    display.setTextSize(1);
    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("PLACA A (EMISSOR)");
    display.display();

    // Configura  o e inicializa  o do DHT11
    pinMode(DHTPIN, INPUT_PULLUP);
    dht.begin();
    delay(2000); // Estabiliza  o do sensor

    pinMode(PINO_AQUECEDOR, OUTPUT);

    pinMode(PINO_RETORNO_ENERGIA, INPUT);

    pinMode(PINO_EXAUSTAO, OUTPUT);
    digitalWrite(PINO_EXAUSTAO, LOW);
    pinMode(PINO_RETORNO_EXAUSTAO, INPUT);

    WiFi.mode(WIFI_STA);
    esp_wifi_set_promiscuous(true);
    esp_wifi_set_channel(11, WIFI_SECOND_CHAN_NONE);
    esp_wifi_set_promiscuous(false);

    if (esp_now_init() == ESP_OK) {
        esp_now_register_recv_cb(AoReceberComando);
        memcpy(peerInfo.peer_addr, macReceptor, 6);
        peerInfo.channel = 11;
        peerInfo.encrypt = false;
        esp_now_add_peer(&peerInfo);
    }
}
```

```
}

void loop() {

    // Desligamento automático temporizado
    if (aquecedorAtivo && (millis() - tempoInicioAquecedor >= DURACAO_AQUECEDOR))
    {

        Serial.println(">>> DESLIGAMENTO AUTOMATICO <<<");

        digitalWrite(PINO_AQUECEDOR, LOW);
        aquecedorAtivo = false;
    }

    if (millis() - ultimoEnvio >= 2000) {
        ultimoEnvio = millis();

        float temp = dht.readTemperature();
        float umid = dht.readHumidity();

        if (!isnan(temp) && !isnan(umid)) {
            dadosEnvio.temperatura = temp;
            dadosEnvio.umidade = umid;
        }

        contadorPacotes++;
        dadosEnvio.contador = contadorPacotes;
        dadosEnvio.energiaOk = digitalRead(PINO_RETORNO_ENERGIA);
        dadosEnvio.exaustaoOK = digitalRead(PINO_RETORNO_EXAUASTAO);

        // Atualiza o conteúdo do Display OLED local no Emissor
        display.clearDisplay();
        display.setCursor(0, 0);
        display.println("PLACA A (EMISSOR)");
        display.setCursor(0, 16);
        display.print("Temp: ");
        display.print(dadosEnvio.temperatura, 1);
        display.println(" C");
        display.setCursor(0, 32);
        display.print("Umid: ");
        display.print(dadosEnvio.umidade, 1);
        display.println(" %");
        display.setCursor(0, 48);
        display.print("Pacote enviado: #");
        display.println(dadosEnvio.contador);
        display.display();

        esp_now_send(macReceptor, (uint8_t *)&dadosEnvio, sizeof(dadosEnvio));
    }
}
```

--- Arquivo: 5.4-receptor_cpp.md ---

```
### 5.4. `src/receptor.cpp`

#include <esp_now.h>
#include <WiFi.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Arduino.h>
#include <ESPAsyncWebServer.h>
#include <LittleFS.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <WiFiClientSecure.h>
#include <SPI.h>
#include <LoRa.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
AsyncWebServer server(80);

volatile bool enviarHostgator = false;
unsigned long ultimoComando = 0;

int canalRoteador = 1;
const float TEMPERATURA_LIGA_EXAUSTAO = 30.0;
const float TEMPERATURA_DESLIGA_EXAUSTAO = 28.0;

uint8_t macEmissor[] = {0xA0, 0xDD, 0x6C, 0x75, 0x0E, 0x14}; // MAC do Emissor

// Struct idÃantica Ã do Emissor
typedef struct struct_mensagem {
    int contador;
    float temperatura;
    float umidade;
    bool energiaOk;
    bool exaustaoOK;
} struct_mensagem;

typedef struct struct_comando {
    bool acionarAquecedor;
    bool ligarExaustao;
} struct_comando;

struct_mensagem dadosRecebidos;
struct_comando comandoEnvio;
esp_now_peer_info_t peerInfo;
bool exaustaoLigada = false;
```

```
void enviarParaHostgator() {
    if (WiFi.status() == WL_CONNECTED) {
        WiFiClientSecure client;
        client.setInsecure(); // Ignora validade do certificado SSL

        HTTPClient http;

        if (http.begin(client, "https://estudecertoja.com.br/salvar_dados.php")) {
            http.addHeader("Content-Type", "application/json");

            float t = isnan(dadosRecebidos.temperatura) ? 0.0 :
dadosRecebidos.temperatura;
            float h = isnan(dadosRecebidos.umidade) ? 0.0 :
dadosRecebidos.umidade;

            String jsonPayload = "{";
            jsonPayload += "\"estacao_id\": \"ESTACAO_01\", ";
            jsonPayload += "\"temperatura\": " + String(t, 1) + ", ";
            jsonPayload += "\"umidade\": " + String(h, 1) + ", ";
            jsonPayload += "\"contador\": " + String(dadosRecebidos.contador) +
            ", ";
            jsonPayload += "\"energia_ok\": " + String(dadosRecebidos.energiaOk ?
"true" : "false") + ", ";
            jsonPayload += "\"exaustao_ligada\": " + String(exaustaoLigada ? "true"
: "false") + ", ";
            jsonPayload += "\"exaustao_ok\": " + String(dadosRecebidos.exaustaoOK ?
"true" : "false") + ", ";
            jsonPayload += "\"aquecedor_ligado\": " +
String(comandoEnvio.acionarAquecedor ? "true" : "false");
            jsonPayload += "}";

            int httpResponseCode = http.POST(jsonPayload);

            Serial.print(">>> RESPOSTA HOSTGATOR (HTTP): ");
            Serial.println(httpResponseCode);

            if (httpResponseCode > 0) {
                String response = http.getString();
                Serial.print("Resposta do Servidor: ");
                Serial.println(response);
            } else {
                Serial.print("[ERRO HTTP] Motivo: ");
                Serial.println(http.errorToString(httpResponseCode).c_str());
            }

            http.end();
        } else {
            Serial.println("[ERRO] Nao foi possivel iniciar a conexao HTTPS");
        }
    }
}

void lerComandoHostgator()
```

```
{
    if (WiFi.status() == WL_CONNECTED)
    {
        WiFiClientSecure client;
        client.setInsecure();

        HTTPClient http;

        if (http.begin(client,
"https://estudecertoja.com.br/zoocontrol/ler_comando.php"))
        {
            int httpCode = http.GET();

            Serial.print("Resposta comando: ");
            Serial.println(httpCode);

            if (httpCode > 0)
            {
                String resposta = http.getString();

                //--- bloco q lêª ultima linha db comandos ---
                JsonDocument doc;
                DeserializationError erro = deserializeJson(doc, resposta);

                if (erro)
                {
                    Serial.println("Erro ao interpretar JSON");
                    http.end();
                    return;
                }

                int idComando = doc["id"].as<int>();
                Serial.print("ID do comando: "); //2 printSerial p confirmar que
idComando recebeu id certo
                Serial.println(idComando);

                if (idComando == 0)
                {
                    http.end();
                    return;
                }

                comandoEnvio.acionarAquecedor = doc["acionar_aquecedor"].as<int>
());

                // comandoEnvio.ligarExaustao = doc["ligar_exaustao"].as<int>();
                //nÃo mudar estado de exaustor pelo banco

                //bloco teste, desligadar apÃs confirmar funcionamento
                Serial.print("***Comando enviado - Aquecedor: ");
                Serial.print(comandoEnvio.acionarAquecedor);
                Serial.print(" | Exaustao: ");
                Serial.println(comandoEnvio.ligarExaustao);
            }
        }
    }
}
```



```

emissor //--- Bloco q envia comando recebido db comandos>net>receptor p

esp_err_t resultado = esp_now_send(macEmissor,
                                   (uint8_t *)&comandoEnvio,
                                   sizeof(comandoEnvio));

Serial.print("ESP-NOW aquecedor: ");
Serial.println(resultado);

if (resultado == ESP_OK)
{
    WiFiClientSecure client;
    client.setInsecure();

    HTTPClient http;

    String url =
"https://estudecertoja.com.br/zoocontrol/executar_comando.php?id=" +
String(idComando);

    if (http.begin(client, url))
    {
        int resposta = http.GET();

        Serial.print("Atualizar comando: ");
        Serial.println(resposta);

        http.end();
    }
}

//-----

Serial.print("Aquecedor: ");
Serial.println(comandoEnvio.acionarAquecedor);

Serial.print("Exaustao: ");
Serial.println(comandoEnvio.ligarExaustao);

//-----

Serial.println("JSON recebido:");
Serial.println(resposta);
}

http.end();
}
}

void AoReceber(const uint8_t *mac_addr, const uint8_t *incomingData, int len) {
    if (len == sizeof(dadosRecebidos)) {
        memcpy(&dadosRecebidos, incomingData, sizeof(dadosRecebidos));
    }
}

```

```
Serial.print("Temp: ");
Serial.print(dadosRecebidos.temperatura, 1);
Serial.print("  ExaustaoLigada: ");
Serial.println(exaustaoLigada);

display.clearDisplay();
display.setCursor(0, 0);
display.println("PLACA B (RECEPTOR)");
display.setCursor(0, 16);
display.print("Temp: ");
display.print(dadosRecebidos.temperatura, 1);
display.println(" C");
display.setCursor(0, 32);
display.print("Umid: ");
display.print(dadosRecebidos.umidade, 1);
display.println(" %");
display.setCursor(0, 48);
display.print("Pacote: #");
display.println(dadosRecebidos.contador);
display.display();

if (!exaustaoLigada && dadosRecebidos.temperatura >=
TEMPERATURA_LIGA_EXAUSTAO)
{
    exaustaoLigada = true;

    comandoEnvio.acionarAquecedor = false;
    Serial.print("Enviando LigarExaustao = ");
    comandoEnvio.ligarExaustao = true;
    Serial.println(comandoEnvio.ligarExaustao);

    esp_now_send(macEmissor,
                  (uint8_t *)&comandoEnvio,
                  sizeof(comandoEnvio));

    Serial.println("Sistema de Exaustao LIGADO");
}
if (exaustaoLigada && dadosRecebidos.temperatura <=
TEMPERATURA_DESLIGA_EXAUSTAO)
{
    exaustaoLigada = false;

    comandoEnvio.acionarAquecedor = false;

    Serial.print("Enviando LigarExaustao = ");
    comandoEnvio.ligarExaustao = false;
    Serial.println(comandoEnvio.ligarExaustao);

    esp_now_send(macEmissor,
                  (uint8_t *)&comandoEnvio,
                  sizeof(comandoEnvio));

    Serial.println("Sistema de Exaustao DESLIGADO");
}
```

```
        enviarHostgator = true;

    }
}

#define LORA_SS    18
#define LORA_RST    14
#define LORA_DIO0    26

void setup() {
    Serial.begin(115200);
    delay(1000); // Tempo para estabilizar a Serial

    Serial.println("Passo 1 Lora");
    SPI.begin(5, 19, 27, 18);
    Serial.println("Passo 2 Lora");
    LoRa.setPins(LORA_SS, LORA_RST, LORA_DIO0);
    Serial.println("Passo 3 Lora");
    if (!LoRa.begin(915E6))
    {
        Serial.println("Falha ao iniciar LoRa");
        while (true);
    }
    Serial.println("LoRa iniciado com sucesso");

    Wire.begin(21, 22);
    display.begin(SSD1306_SWITCHCAPVCC, 0x3C);
    display.setTextColor(WHITE);
    display.setTextSize(1);

    display.clearDisplay();
    display.setCursor(0, 0);
    display.println("Conectando Wi-Fi...");
    display.display();

    if (!LittleFS.begin(true)) {
        Serial.println("Erro ao montar o LittleFS!");
    }

    // Tenta conectar no Wi-Fi Residencial
    WiFi.mode(WIFI_AP_STA);
    WiFi.begin("NILSON 2.4", "81111270in"); // Mantido exatamente como no
documento

    int tentativas = 0;
    while (WiFi.status() != WL_CONNECTED && tentativas < 20) {
        delay(500);
        Serial.print(".");
        tentativas++;
    }

    display.clearDisplay();
    display.setCursor(0, 0);
```

```

if (WiFi.status() == WL_CONNECTED) {
    canalRoteador = WiFi.channel();

    Serial.println("\n>>> WiFi conectado <<<");
    Serial.print("IP Receptor: ");
    Serial.println(WiFi.localIP());
    Serial.print("Canal WiFi Roteador: ");
    Serial.println(canalRoteador);

    // Exibe IP e Canal no OLED
    display.println("WIFI CONECTADO!");
    display.setCursor(0, 16);
    display.print("IP: ");
    display.println(WiFi.localIP());
    display.setCursor(0, 32);
    display.print("Canal: ");
    display.println(canalRoteador);
} else {
    Serial.println("\n[ERRO] Falha ao conectar no Wi-Fi!");
    display.println("FALHA NO WI-FI!");
    display.setCursor(0, 16);
    display.println("Verifique SSID/Senha");
}
display.display();

if (esp_now_init() == ESP_OK) {
    esp_now_register_recv_cb(AoReceber);
    memcpy(peerInfo.peer_addr, macEmissor, 6);
    peerInfo.channel = canalRoteador;
    peerInfo.encrypt = false;
    esp_now_add_peer(&peerInfo);
}

server.on("/", HTTP_GET, [](AsyncWebServerRequest *request) {
    request->send(LittleFS, "/index.html", "text/html");
});
server.on("/style.css", HTTP_GET, [](AsyncWebServerRequest *request) {
    request->send(LittleFS, "/style.css", "text/css");
});
server.on("/script.js", HTTP_GET, [](AsyncWebServerRequest *request) {
    request->send(LittleFS, "/script.js", "text/javascript");
});

server.on("/dados", HTTP_GET, [](AsyncWebServerRequest *request) {
    float t = isnan(dadosRecebidos.temperatura) ? 0.0 :
dadosRecebidos.temperatura;
    float h = isnan(dadosRecebidos.umidade) ? 0.0 : dadosRecebidos.umidade;

    String json = "{";
    json += "\"temperatura\":" + String(t, 1) + ",";
    json += "\"umidade\":" + String(h, 1) + ",";
    json += "\"contador\":" + String(dadosRecebidos.contador) + ",";
    json += "\"energia_ok\":" + String(dadosRecebidos.energiaOk ? "true" :

```

```

"false");
    json += ",\\"exaustao_ligada\":" + String(exaustaoLigada ? "true" :
"false");
    json += ",\\"exaustao_ok\":" + String(dadosRecebidos.exaustaoOK ? "true" :
"false");
    json += "}";

    request->send(200, "application/json", json);
});

server.on("/ligar-aquecedor", HTTP_POST, [](AsyncWebServerRequest *request) {
    comandoEnvio.acionarAquecedor = true;
    esp_err_t result = esp_now_send(macEmissor, (uint8_t *)&comandoEnvio,
sizeof(comandoEnvio));

    if (result == ESP_OK) {
        request->send(200, "text/plain", "Comando enviado com sucesso");
    } else {
        request->send(500, "text/plain", "Erro ao enviar via ESP-NOW");
    }
});

server.begin();
}

void loop()
{
    // Teste de comunica  o Lora
    // int packetSize = LoRa.parsePacket();
    // if (packetSize)
    // {
    //     Serial.print("Recebido: ");
    //     while (LoRa.available())
    //     {
    //         Serial.print((char)LoRa.read());
    //     }
    //     Serial.println();
    // }

    if (millis() - ultimoComando >= 5000)
    {
        ultimoComando = millis();
        lerComandoHostgator();
    }

    if (enviarHostgator)
    {
        enviarHostgator = false;
        enviarParaHostgator();
    }
}

```

--- Arquivo: 5.5-Web.md ---

```
### 5.5. `WEB`

#### 5.5.1. `data/index.html`
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Painel ESP32</title>
  <link rel="stylesheet" type="text/css" href="style.css">
  <script src="script.js" defer></script>
</head>
<body>
  <h1>Estação de Sensores</h1>

  <div class="card">
    <p>Temperatura</p>
    <div class="valor"><span id="temp">--</span> °C</div>
  </div>

  <div class="card">
    <p>Umidade</p>
    <div class="valor"><span id="umid">--</span> %</div>
  </div>

  <div class="card">
    <h3>Controle do Aquecedor</h3>
    <button id="btnAquecedor" onclick="acionarAquecedor()">Ligar Aquecedor
(30s)</button>
    <div class="status-container">
      <span>Retorno de Energia:</span>
      <span id="ledStatus" class="led desligado"></span>
    </div>
  </div>

  <div class="card">
    <h3>Sistema de Exaustão</h3>
    <span>Estado:</span>
    <span id="ledExaustao" class="led desligado"></span>
    <span id="textoExaustao">Desligado</span>
  </div>

  <p>Pacotes recebidos: <b id="contador">0</b></p>
</body>
</html>
```

```
#### 5.5.2. `data/style.css`

body {
  font-family: Arial, sans-serif;
  text-align: center;
  margin-top: 30px;
  background-color: #f4f4f9;
}
h1 {
  color: #333;
}
.card {
  background: white;
  padding: 20px;
  margin: 10px auto;
  width: 260px;
  border-radius: 10px;
  box-shadow: 0 2px 5px rgba(0,0,0,0.2);
}
.valor {
  font-size: 2em;
  color: #007bff;
  font-weight: bold;
}
button {
  background-color: #28a745;
  color: white;
  border: none;
  padding: 12px 20px;
  font-size: 1em;
  border-radius: 5px;
  cursor: pointer;
  transition: 0.3s;
}
button:disabled {
  background-color: #6c757d;
  cursor: not-allowed;
}
.status-container {
  margin-top: 15px;
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 10px;
}
.led {
  width: 16px;
  height: 16px;
  border-radius: 50%;
  display: inline-block;
  background-color: #ccc;
}
```

```
.led.ligado {
  background-color: #dc3545; /* Bolinha vermelha quando ativado */
  box-shadow: 0 0 8px #dc3545;
}
.led.desligado {
  background-color: #6c757d;
}
```

5.5.3. `data/script.js`

```
function atualizarDados() {

  const ledExaustao = document.getElementById('ledExaustao');
  const textoExaustao = document.getElementById('textoExaustao');

  fetch('/dados?t=' + new Date().getTime(), { cache: 'no-store' })
    .then(response => {
      if (!response.ok) throw new Error('Erro na resposta');
      return response.json();
    })
    .then(data => {
      // Atualiza os valores na tela
      document.getElementById('temp').innerText = data.temperatura;
      document.getElementById('umid').innerText = data.umidade;
      document.getElementById('contador').innerText = data.contador;

      // Atualiza o LED de retorno de energia
      const led = document.getElementById('ledStatus');
      if (data.energia_ok) {
        led.className = 'led ligado';
      } else {
        led.className = 'led desligado';
      }

      if (data.exaustao_ok) {
        ledExaustao.className = "led ligado";
        textoExaustao.innerText = "Ligado";
      } else {
        ledExaustao.className = "led desligado";
        textoExaustao.innerText = "Desligado";
      }
    })
    .catch(err => console.log('Aguardando dados...', err));
}

function acionarAquecedor() {
  const btn = document.getElementById('btnAquecedor');
  if (btn) btn.disabled = true;
}
```



```

    fetch('/ligar-aquecedor', {
      method: 'POST',
      cache: 'no-store'
    })
    .then(response => {
      console.log('Comando enviado com sucesso');
      // Força uma atualização imediata do painel após o clique
      atualizarDados();
    })
    .catch(err => console.log('Erro ao acionar:', err))
    .finally(() => {
      // Reabilita o botão após 1 segundo
      setTimeout(() => {
        if (btn) btn.disabled = false;
      }, 1000);
    });
  }

// Garante o loop contínuo de atualização a cada 2 segundos
setInterval(atualizarDados, 2000);

// Executa a primeira leitura assim que a página carrega
document.addEventListener('DOMContentLoaded', atualizarDados);

```

5.5.4. `PHP`

5.5.4.1. `comando.php`

```

<?php

$servername = "localhost";
$username   = "estude43_userNilson";
$password   = "GjTX@@jqyD@E";
$dbname     = "estude43_estacao_db";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die("Erro conexão");
}

$acao = $_GET["acao"] ?? "";

if ($acao == "ligar_aquecedor") {

    $sql = "INSERT INTO comandos (acionar_aquecedor, ligar_exaustao)
            VALUES (TRUE, FALSE)";

    $conn->query($sql);

```

```
        echo "Comando criado";
    }

    $conn->close();

?>
```

5.5.4.2. `dados.php`

```
<?php

$servername = "localhost";
$username   = "estude43_userNilson";
$password   = "GjTX@@jqyD@E";
$dbname     = "estude43_estacao_db";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    http_response_code(500);
    die("Erro na conexão");
}

$sql = "SELECT
        temperatura,
        umidade,
        contador,
        energia_ok,
        exaustao_ligada,
        exaustao_ok,
        aquecedor_ligado
    FROM leituras_sensores
    ORDER BY id DESC
    LIMIT 1";

$result = $conn->query($sql);

if ($result->num_rows > 0) {

    $dados = $result->fetch_assoc();

    header('Content-Type: application/json');

    echo json_encode($dados);

} else {

    http_response_code(404);
```

```
        echo json_encode([
            "erro" => "Nenhum dado encontrado"
        ]);
    }

    $conn->close();

?>
```

5.5.4.3. `executar\_comando.php`

```
<?php

$servername = "localhost";
$username   = "estude43_userNilson";
$password   = "GjTX@@jqyD@E";
$dbname     = "estude43_estacao_db";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die("Erro conexão");
}

$id = intval($_GET["id"] ?? 0);

if ($id > 0) {

    $sql = "UPDATE comandos
            SET executando = 1
            WHERE id = $id";

    $conn->query($sql);

    echo "OK";
}

$conn->close();

?>
```

5.5.4.4. `executar\_comando.php`

```
<?php

error_reporting(E_ALL);
ini_set('display_errors', 1);

$servername = "localhost";
$username    = "estude43_userNilson";
```

```
$password = "GjTX@@jqyD@E";
$dbname = "estude43_estacao_db";

$conn = new mysqli($servername, $username, $password, $dbname);

if ($conn->connect_error) {
    die("Erro conexao");
}

$sql = "SELECT
        id,
        acionar_aquecedor,
        ligar_exaustao
    FROM comandos
    WHERE executando = FALSE
    ORDER BY id ASC
    LIMIT 1";

$result = $conn->query($sql);

header('Content-Type: application/json');

if ($result->num_rows > 0) {

    echo json_encode($result->fetch_assoc());

} else {

    echo json_encode([
        "id" => 0,
        "acionar_aquecedor" => false,
        "ligar_exaustao" => false
    ]);

}

$conn->close();

?>
```